

Client Data Integration & Analytics - MySQL

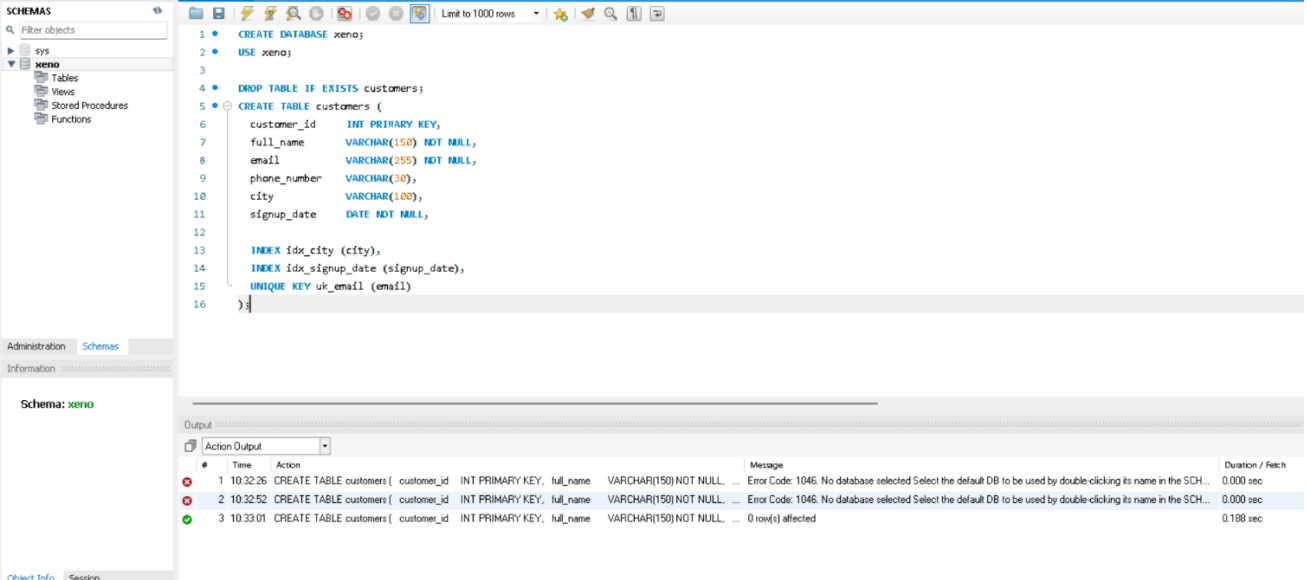
Objective: Understand Your Thought Process on Client Data Handling

Assignment Tasks:

1. SQL & Data Familiarity:

1. What steps would you take to review this data before importing it into a system? Please explain the process in not more than 3 lines.

Before importing the CSV, I quickly validate whether the file has all required columns, correct data types, and no duplicate customer IDs/emails. I then standardise fields (clean phone numbers, normalise city names, lowercase email addresses) and manually convert DD-MM-YYYY to MySQL-compatible YYYY-MM-DD in Excel.

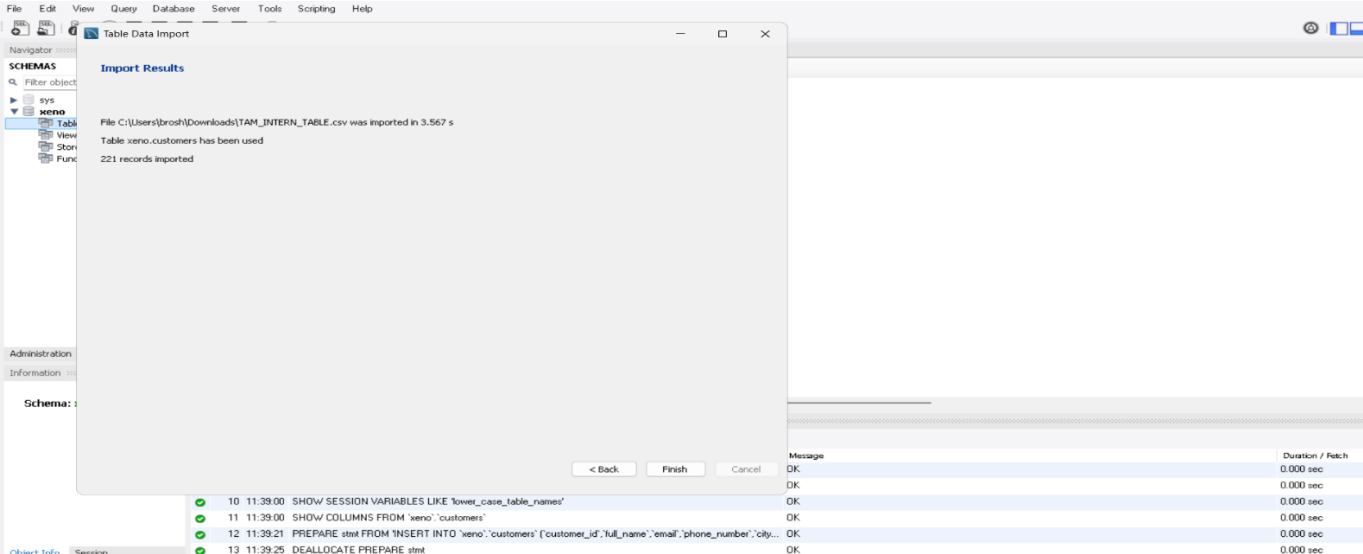


The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'sys' and 'xeno' databases. The main editor shows a SQL script:

```
1 CREATE DATABASE xeno;
2 USE xeno;
3
4 DROP TABLE IF EXISTS customers;
5 CREATE TABLE customers (
6   customer_id INT PRIMARY KEY,
7   full_name VARCHAR(150) NOT NULL,
8   email VARCHAR(255) NOT NULL,
9   phone_number VARCHAR(30),
10  city VARCHAR(100),
11  signup_date DATE NOT NULL,
12
13  INDEX idx_city (city),
14  INDEX idx_signup_date (signup_date),
15  UNIQUE KEY uk_email (email)
16 );
```

The 'Output' window at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:32:25	CREATE TABLE customers (customer_id INT PRIMARY KEY, full_name VARCHAR(150) NOT NULL, ...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.000 sec
2	10:32:52	CREATE TABLE customers (customer_id INT PRIMARY KEY, full_name VARCHAR(150) NOT NULL, ...	Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCH...	0.000 sec
3	10:33:01	CREATE TABLE customers (customer_id INT PRIMARY KEY, full_name VARCHAR(150) NOT NULL, ...	0 row(s) affected	0.188 sec



The screenshot shows the 'Table Data Import' dialog box open over the MySQL Workbench interface. The dialog displays the following information:

- File: C:\Users\brosh\Downloads\TAM_INTERM_TABLE.csv was imported in 3.567 s
- Table xeno.customers has been used
- 221 records imported

The 'Import Results' window at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
10	11:39:00	SHOW SESSION VARIABLES LIKE 'lower_case_table_names'	OK	0.000 sec
11	11:39:00	SHOW COLUMNS FROM 'xeno'.customers	OK	0.000 sec
12	11:39:21	PREPARE stmt FROM 'INSERT INTO 'xeno'.customers' ('customer_id','full_name','email','phone_number','city',...	OK	0.000 sec
13	11:39:25	DEALLOCATE PREPARE stmt	OK	0.000 sec

2. Please execute each of the queries and support your queries with a screenshot of your results.

Write SQL queries for the following:

- Write a query to display all customers from the city 'Delhi'.

```
SELECT *  
FROM customers  
WHERE city = 'Delhi';
```

The screenshot shows the SQL Studio interface. The SQL Editor contains the following query:

```
14 INDEX idx_signup_date (signup_date),  
15 UNIQUE KEY uk_email (email)  
16 )  
17 -- 2.1. All customers from 'Delhi'  
18 SELECT *  
19 FROM customers  
20 WHERE city = 'Delhi';
```

The Result Grid shows the following data:

customer_id	full_name	email	phone_number	city	signup_date
156259	Hrishita Gopal	dividra@hotmail.com	7344359134	Delhi	2024-08-28
159463	Divij Viswanathan	jwarrior@yahoo.com	9460578167	Delhi	2023-11-02
191011	Samarth Subramanian	jvn90@gmail.com	8340412987	Delhi	2023-09-10
196722	Adah Yogi	psoman@yahoo.com	8591581358	Delhi	2025-03-27
245018	Kabir Vaidya	rhanda@bal.org	8491731479	Delhi	2023-05-11
247923	Rania Bhattacharyya	devansh9@hotmail.com	8893549828	Delhi	2025-04-04
250616	Nishith Ramchandran	gobman@chacko.net	7822335748	Delhi	2024-06-13
261495	Pihu Tador	farhan12@chandra.com	7864389575	Delhi	2023-10-01
267289	Umang Kaur	hdara@gmail.com	8681162759	Delhi	2025-03-27
378194	Kavya Kadalia	golavayantar@gmail.com	7762967030	Delhi	2025-03-21
433818	Yasmin Dora	abonra@dhawan.com	9248283582	Delhi	2023-12-30
451453	Aarush Uppal	vrikabhargava@gmail.com	8927631712	Delhi	2024-05-19

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
9	11:38:52	SHOW DATABASES	OK	0.000 sec
10	11:39:00	SHOW SESSION VARIABLES LIKE 'lower_case_table_names'	OK	0.000 sec
11	11:39:00	SHOW COLUMNS FROM 'xeno'.customers	OK	0.000 sec
12	11:39:21	PREPARE stmt FROM 'INSERT INTO 'xeno'.customers ('customer_id','full_name','email','phone_number','ci...'	OK	0.000 sec
13	11:39:25	DEALLOCATE PREPARE stmt	OK	0.000 sec
14	11:48:44	SELECT * FROM customers WHERE city = 'Delhi' LIMIT 0, 1000	31 row(s) returned	0.016 sec / 0.000 sec

- Count the number of signups in the last 30 days. Assume today to be 16th April 2025.

```
SELECT COUNT(*) AS signups_last_30_days  
FROM customers  
WHERE signup_date >= DATE('2025-04-16') - INTERVAL 30 DAY  
AND signup_date <= DATE('2025-04-16');
```

The screenshot shows the SQL Studio interface. The SQL Editor contains the following query:

```
20 WHERE city = 'Delhi';  
21  
22 -- 2.2. Signups in the last 30 days (Assume from 2025-04-16)  
23 SELECT COUNT(*) AS signups_last_30_days  
24 FROM customers  
25 WHERE signup_date >= DATE('2025-04-16') - INTERVAL 30 DAY  
26 AND signup_date <= DATE('2025-04-16');
```

The Result Grid shows the following data:

signups_last_30_days
65

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
10	11:39:00	SHOW SESSION VARIABLES LIKE 'lower_case_table_names'	OK	0.000 sec
11	11:39:00	SHOW COLUMNS FROM 'xeno'.customers	OK	0.000 sec
12	11:39:21	PREPARE stmt FROM 'INSERT INTO 'xeno'.customers ('customer_id','full_name','email','phone_number','ci...'	OK	0.000 sec
13	11:39:25	DEALLOCATE PREPARE stmt	OK	0.000 sec
14	11:48:44	SELECT * FROM customers WHERE city = 'Delhi' LIMIT 0, 1000	31 row(s) returned	0.016 sec / 0.000 sec
15	11:59:34	SELECT COUNT(*) AS signups_last_30_days FROM customers WHERE signup_date >= DATE('2025-04-16') ...	1 row(s) returned	0.016 sec / 0.000 sec

- List unique cities where customers are based.

```
SELECT DISTINCT city
FROM customers
ORDER BY city;
```

The screenshot shows a SQL IDE interface. The left sidebar displays a 'SCHEMAS' tree with 'sys' and 'xeno' (containing Tables, Views, Stored Procedures, and Functions). The main editor shows a SQL script with a filter for 'signup_date' and a comment '-- 3.3. Unique cities'. The script is:
WHERE signup_date >= DATE('2025-04-16') - INTERVAL 30 DAY
AND signup_date <= DATE('2025-04-16');
-- 3.3. Unique cities
SELECT DISTINCT city
FROM customers
ORDER BY city;
The 'Result Grid' shows a list of cities: Ahmedabad, Bangalore, Chennai, Delhi, Hyderabad, Kolkata, Mumbai, and Pune. The 'Output' pane shows the execution log, including the final query execution: 'SELECT DISTINCT city FROM customers ORDER BY city LIMIT 0, 1000' which returned 8 rows in 0.094 seconds.

- List the top 3 cities by number of signups.

```
SELECT city, COUNT(*) AS signup_count
FROM customers
GROUP BY city
ORDER BY signup_count DESC
LIMIT 3;
```

The screenshot shows the same SQL IDE interface. The main editor shows a SQL script with a comment '-- 4.4. Top 3 cities by number of signups'. The script is:
SELECT city, COUNT(*) AS signup_count
FROM customers
GROUP BY city
ORDER BY signup_count DESC
LIMIT 3;
The 'Result Grid' shows the top 3 cities by signup count: Kolkata (32), Pune (32), and Bangalore (31). The 'Output' pane shows the execution log, including the final query execution: 'SELECT city, COUNT(*) AS signup_count FROM customers GROUP BY city ORDER BY signup_count DESC ...' which returned 3 rows in 0.000 seconds.

- Assume there's another table orders (customer_id, order_id, amount).
How would you find customers who have never placed an order?

```
CREATE TABLE orders (  
    customer_id INT NOT NULL,  
    order_id INT PRIMARY KEY,  
    amount DECIMAL(10,2) NOT NULL  
);
```

```
INSERT INTO orders (customer_id, order_id, amount) VALUES  
(1, 10001, 950.00),  
(2, 10002, 1299.00),  
(3, 10003, 499.00);
```

```
-- 1.5. Customers who never placed an order (orders: customer_id,  
order_id, amount)  
SELECT c.*  
FROM customers AS c  
LEFT JOIN orders AS o  
ON o.customer_id = c.customer_id;
```

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with 'sys' and 'xeno' databases. The 'xeno' database is expanded, showing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The 'Tables' folder is selected. The main pane shows a query window with the following SQL code:

```
31 ORDER BY city;  
32  
33 -- 4.4. Top 3 cities by number of signups  
34 * SELECT city, COUNT(*) AS signup_count  
35 FROM customers  
36 GROUP BY city  
37 ORDER BY signup_count DESC  
38 LIMIT 3;  
39  
40 * CREATE TABLE orders (  
41     customer_id INT NOT NULL,  
42     order_id INT PRIMARY KEY,  
43     amount DECIMAL(10,2) NOT NULL  
44 );  
45  
46 -- Example  
47 * INSERT INTO orders (customer_id, order_id, amount) VALUES  
48 (1, 10001, 950.00),  
49 (2, 10002, 1299.00),  
50 (3, 10003, 499.00);
```

The bottom pane shows the 'Output' window with the 'Action Output' tab selected. It displays the execution results of the SQL code, including the number of rows returned and the duration of each operation.

#	Time	Action	Message	Duration / Fetch
14	11:48:44	SELECT * FROM customers WHERE city = 'Delhi' LIMIT 0, 1000	31 row(s) returned	0.016 sec / 0.000 sec
15	11:59:34	SELECT COUNT(*) AS signups_last_30_days FROM customers WHERE signup_date >= DATE('2025-04-16') ...	1 row(s) returned	0.016 sec / 0.000 sec
16	13:25:22	SELECT DISTINCT city FROM customers ORDER BY city LIMIT 0, 1000	8 row(s) returned	0.094 sec / 0.000 sec
17	13:27:08	SELECT city, COUNT(*) AS signup_count FROM customers GROUP BY city ORDER BY signup_count DESC ...	3 row(s) returned	0.000 sec / 0.000 sec
18	13:32:37	CREATE TABLE orders (customer_id INT NOT NULL, order_id INT PRIMARY KEY, amount DECIM...	0 row(s) affected	0.171 sec
19	13:32:37	INSERT INTO orders (customer_id, order_id, amount) VALUES (1, 10001, 950.00), (2, 10002, 1299.00), (3, 10...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec

Navigator

SQL File 4

Filter objects

sys

xeno

Tables

Views

Stored Procedures

Functions

Administration

Schemas

Information

No object selected

Object Info

Session

Limit to 1000 rows

50 (3, 1000, 499.00);

51

52 -- 2.5. Customers who never placed an order (orders: customer_id, order_id, amount)

53 SELECT c.*

54 FROM customers AS c

55 LEFT JOIN orders AS o

56 ON o.customer_id = c.customer_id

Result Grid

customer_id	full_name	email	phone_number	city	signup_date
105394	Eshani Kota	bhavinbhargava@hotmail.com	8151619403	Bangalore	2023-05-05
107397	Ojas Devan	stuvan83@gmail.com	9542793419	Bangalore	2025-04-08
109407	Miraya Sura	mannatra@yahoo.com	9254427086	Pune	2023-05-11
110995	Mehul Balan	rajagopalani@sara@hotmail.com	7202585038	Pune	2025-03-27
122197	Yuvraj Srinivas	tarini75@hotmail.com	7825021492	Hyderabad	2024-01-07
127033	Myra Keer	jayaramanaman@gmail.com	9258804475	Bangalore	2023-08-01
128097	Jayesh Jha	urvi41@yahoo.com	9834699453	Kolkata	2025-03-24
132742	Siya Shankar	omaharaj@grover.com	8027015672	Mumbai	2024-02-04
137770	Jayesh Bhandari	ritvikloyal@yahoo.com	7209194799	Hyderabad	2023-05-12
156259	Hrshita Gopal	divijdra@hotmail.com	7344359134	Delhi	2024-08-28
156523	Miraan Lata	lhans@yahoo.com	8762147286	Mumbai	2024-05-03
159463	Divij Yiswanathan	jwanior@yahoo.com	9460578167	Delhi	2023-11-02

Result 5

Output

Action Output

#	Time	Action	Message	Duration / Fetch
15	11:59:34	SELECT COUNT(*) AS signups_last_30_days FROM customers WHERE signup_date >= DATE(2025-04-16) ...	1 row(s) returned	0.016 sec / 0.000 sec
16	13:25:22	SELECT DISTINCT city FROM customers ORDER BY city LIMIT 0, 1000	8 row(s) returned	0.094 sec / 0.000 sec
17	13:27:08	SELECT city, COUNT(*) AS signup_count FROM customers GROUP BY city ORDER BY signup_count DESC ...	3 row(s) returned	0.000 sec / 0.000 sec
18	13:32:37	CREATE TABLE orders (customer_id INT NOT NULL, order_id INT PRIMARY KEY, amount DECIM...	0 row(s) affected	0.171 sec
19	13:32:37	INSERT INTO orders (customer_id, order_id, amount) VALUES (1, 10001, 950.00), (2, 10002, 1299.00), (3, 10...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec
20	13:35:42	SELECT c.* FROM customers AS c LEFT JOIN orders AS o ON o.customer_id = c.customer_id LIMIT 0, 1000	221 row(s) returned	0.000 sec / 0.000 sec

2. Data Transformation & Enrichment:

The operations teams want a few more details to ensure smooth operations.

Questions:

- Add a new column to show if the email domain is 'gmail.com' or not. Fill it with 'Yes' or 'No'.

ALTER TABLE customers

ADD COLUMN is_gmail ENUM('Yes','No') NULL;

UPDATE customers

SET is_gmail = CASE

WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN

'Yes'

ELSE 'No'

END;

Navigator

SQL File 4

Filter objects

sys

xeno

Tables

customers

orders

Views

Stored Procedures

Functions

Administration

Schemas

Information

No object selected

Object Info

Session

Limit to 1000 rows

48 (1, 10001, 950.00),

49 (2, 10002, 1299.00),

50 (3, 10003, 499.00);

51

52 -- 1.5. Customers who never placed an order (orders: customer_id, order_id, amount)

53 SELECT c.*

54 FROM customers AS c

55 LEFT JOIN orders AS o

56 ON o.customer_id = c.customer_id

57

58 -- 2.1. Flag Gmail emails

59 ALTER TABLE customers

60 ADD COLUMN is_gmail ENUM('Yes','No') NULL;

61

62 UPDATE customers

63 SET is_gmail = CASE

64 WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN 'Yes'

65 ELSE 'No'

66 END;

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:53:14	UPDATE customers SET is_gmail = CASE WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' T...	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.031 sec

Query Completed

- **Extract the first name from the name column and store it in 'first_name'**

```
ALTER TABLE customers  
ADD COLUMN first_name VARCHAR(100);
```

```
UPDATE customers  
SET first_name = SUBSTRING_INDEX(full_name, ' ', 1);
```

The screenshot shows a SQL IDE interface. On the left is a 'SCHEMAS' pane with a tree view showing 'sys', 'xeno', 'Tables' (customers, orders), 'Views', 'Stored Procedures', and 'Functions'. The main editor displays a SQL script with line numbers 56 to 74. The script includes a join condition, a comment about flagging Gmail emails, an ALTER TABLE to add an 'is_gmail' column, an UPDATE to set 'is_gmail' based on the email domain, another comment about extracting the first name, and finally an ALTER TABLE to add a 'first_name' column followed by an UPDATE to populate it from 'full_name'. The 'Output' pane at the bottom shows the execution results of these three statements, indicating successful completion for each.

#	Time	Action	Message	Duration / Fetch
1	13:53:14	UPDATE customers SET is_gmail = CASE WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN 'Yes'	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.031 sec
2	13:55:31	ALTER TABLE customers ADD COLUMN first_name VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
3	13:55:31	UPDATE customers SET first_name = SUBSTRING_INDEX(full_name, ' ', 1)	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.015 sec

- **Add a column 'signup_month' to capture the month name the customer signed up.**

```
ALTER TABLE customers  
ADD COLUMN signup_month VARCHAR(20);
```

```
UPDATE customers  
SET signup_month = DATE_FORMAT(signup_date, '%M');
```

SQL File 4

```

63 SET is_gmail = CASE
64 WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN 'Yes'
65 ELSE 'No'
66 END;
67
68 -- 2.2. Extract first name
69 ALTER TABLE customers
70 ADD COLUMN first_name VARCHAR(100);
71
72 UPDATE customers
73 SET first_name = SUBSTRING_INDEX(full_name, ' ', 1);
74
75 -- 2.3. Month name of signup
76 ALTER TABLE customers
77 ADD COLUMN signup_month VARCHAR(20);
78
79 UPDATE customers
80 SET signup_month = DATE_FORMAT(signup_date, '%M');
81

```

Output

#	Time	Action	Message	Duration / Fetch
1	13:53:14	UPDATE customers SET is_gmail = CASE WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN 'Yes' ELSE 'No' END;	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.031 sec
2	13:55:31	ALTER TABLE customers ADD COLUMN first_name VARCHAR(100);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
3	13:55:31	UPDATE customers SET first_name = SUBSTRING_INDEX(full_name, ' ', 1);	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.015 sec
4	13:56:34	ALTER TABLE customers ADD COLUMN signup_month VARCHAR(20);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
5	13:56:34	UPDATE customers SET signup_month = DATE_FORMAT(signup_date, '%M');	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.016 sec

- Create a report that shows the no. of GMAIL customers who have signed up for each day of the week.

```

SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS
gmail_signups
FROM customers
WHERE is_gmail = 'Yes'
GROUP BY DAYNAME(signup_date)
ORDER BY gmail_signups DESC;

```

SQL File 4

```

81
82 -- 2.4. Gmail signups by day of week
83 SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups
84 FROM customers
85 WHERE is_gmail = 'Yes'
86 GROUP BY DAYNAME(signup_date)
87 ORDER BY gmail_signups DESC;

```

Result Grid

signup_dow	gmail_signups
Friday	9
Sunday	7
Tuesday	6
Monday	5
Wednesday	4
Saturday	2
Thursday	1

Output

#	Time	Action	Message	Duration / Fetch
1	13:53:14	UPDATE customers SET is_gmail = CASE WHEN LOWER(SUBSTRING_INDEX(email, '@', -1)) = 'gmail.com' THEN 'Yes' ELSE 'No' END;	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.031 sec
2	13:55:31	ALTER TABLE customers ADD COLUMN first_name VARCHAR(100);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
3	13:55:31	UPDATE customers SET first_name = SUBSTRING_INDEX(full_name, ' ', 1);	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.015 sec
4	13:56:34	ALTER TABLE customers ADD COLUMN signup_month VARCHAR(20);	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
5	13:56:34	UPDATE customers SET signup_month = DATE_FORMAT(signup_date, '%M');	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.016 sec
6	13:58:02	SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups FROM customers WHERE is_gmail = 'Yes' GROUP BY DAYNAME(signup_date) ORDER BY gmail_signups DESC;	7 row(s) returned	0.000 sec / 0.000 sec

- Create a new table 'vip_customers' with customers from Delhi, Mumbai and Bangalore who have signed up in the last 60 days from 16th April 2025.

```
CREATE TABLE vip_customers AS
SELECT *
FROM customers
WHERE city IN ('Delhi','Mumbai','Bangalore')
AND signup_date >= DATE('2025-04-16') - INTERVAL 60 DAY
AND signup_date <= DATE('2025-04-16');

SELECT COUNT(*) AS vip_count
FROM vip_customers;
SELECT * FROM vip_customers
LIMIT 20;
```

The screenshot shows the SQL Studio interface with the following SQL queries executed:

```
77 ADD COLUMN signup_month VARCHAR(20);
78
79 UPDATE customers
80 SET signup_month = DATE_FORMAT(signup_date, '%M');
81
82 -- 2.4. Gmail signups by day of week
83 SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups
84 FROM customers
85 WHERE is_gmail = 'Yes'
86 GROUP BY DAYNAME(signup_date)
87 ORDER BY gmail_signups DESC;
88
89 -- TS. VIPs: Delhi/Mumbai/Bangalore in last 60 days from 2025-04-16
90 CREATE TABLE vip_customers AS
91 SELECT *
92 FROM customers
93 WHERE city IN ('Delhi','Mumbai','Bangalore')
94 AND signup_date >= DATE('2025-04-16') - INTERVAL 60 DAY
95 AND signup_date <= DATE('2025-04-16');
```

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
2	13:55:31	ALTER TABLE customers: ADD COLUMN first_name VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.110 sec
3	13:55:31	UPDATE customers SET first_name = SUBSTRING_INDEX(last_name, ' ', 1)	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.015 sec
4	13:56:34	ALTER TABLE customers: ADD COLUMN signup_month VARCHAR(20)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
5	13:56:34	UPDATE customers SET signup_month = DATE_FORMAT(signup_date, '%M')	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.016 sec
6	13:58:02	SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups FROM customers WHERE is...	7 row(s) returned	0.000 sec / 0.000 sec
7	14:02:29	CREATE TABLE vip_customers AS SELECT * FROM customers WHERE city IN ('Delhi','Mumbai','Bangalore') ...	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.062 sec

The screenshot shows the SQL Studio interface with the following SQL queries executed:

```
92 FROM customers
93 WHERE city IN ('Delhi','Mumbai','Bangalore')
94 AND signup_date >= DATE('2025-04-16') - INTERVAL 60 DAY
95 AND signup_date <= DATE('2025-04-16');
96
97 SELECT COUNT(*) AS vip_count FROM vip_customers;
98 SELECT * FROM vip_customers LIMIT 20;
```

The Result Grid shows the following data:

customer_id	full_name	email	phone_number	city	signup_date	is_gmail	first_name	signup_month
107397	Ojas Devan	stuvan83@gmail.com	9542793419	Bangalore	2025-04-08	Yes	Ojas	April
196722	Adah Yogi	psoman@yahoo.com	8591581358	Delhi	2025-03-27	No	Adah	March
224232	Divyansh Tripathi	jyohannan@datta.com	8647981998	Mumbai	2025-03-22	No	Divyansh	March
240679	Ahana Kumar	chirag63@sabharwal.com	8580016724	Mumbai	2025-03-18	No	Ahana	March
247923	Rania Bhattacharyya	devansh39@hotmail.com	8893549828	Delhi	2025-04-04	No	Rania	April
260398	Rasha Seshadri	tejasbhandari@yahoo.com	8496435124	Bangalore	2025-03-22	No	Rasha	March
266733	Adah Kamdar	gokul03@gmail.com	8643563127	Mumbai	2025-04-11	Yes	Adah	April
276289	Umayang Kaur	hdsra@gmail.com	8681162759	Delhi	2025-03-27	Yes	Umayang	March
285490	Lakshay Loke	lcheema@sharaf.com	9040520243	Bangalore	2025-04-14	No	Lakshay	April
378194	Kavya Kadalia	golanayantrara@gmail.com	7762967030	Delhi	2025-03-21	Yes	Kavya	March
402031	Urvi Tiwari	kabirhegde@yahoo.com	9973478222	Bangalore	2025-04-03	No	Urvi	April

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
4	13:56:34	ALTER TABLE customers: ADD COLUMN signup_month VARCHAR(20)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
5	13:56:34	UPDATE customers SET signup_month = DATE_FORMAT(signup_date, '%M')	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.016 sec
6	13:58:02	SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups FROM customers WHERE is...	7 row(s) returned	0.000 sec / 0.000 sec
7	14:02:29	CREATE TABLE vip_customers AS SELECT * FROM customers WHERE city IN ('Delhi','Mumbai','Bangalore') ...	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.062 sec
8	14:05:19	SELECT COUNT(*) AS vip_count FROM vip_customers LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
9	14:05:19	SELECT * FROM vip_customers LIMIT 20	20 row(s) returned	0.000 sec / 0.000 sec

3. Analytics & Reporting:

The business team wants insights from the customer data.

Questions:

- Show a monthly signup count for the past 6 months.

```
SELECT DATE_FORMAT(signup_date, '%Y-%m') AS month, COUNT(*) AS
signup_count
FROM customers
WHERE signup_date >= DATE('2025-04-16') - INTERVAL 6 MONTH
AND signup_date <= DATE('2025-04-16')
GROUP BY DATE_FORMAT(signup_date, '%Y-%m')
ORDER BY month;
```

The screenshot shows a SQL IDE interface. On the left is a 'SCHEMAS' sidebar with a tree view containing 'sys', 'xeno', 'Tables' (customers, orders), 'Views', 'Stored Procedures', and 'Functions'. The main editor displays a SQL query (lines 100-106) for a monthly signup count. Below the editor is a 'Result Grid' showing the query results. At the bottom, an 'Action Output' pane shows the execution log.

month	signup_count
2024-10	4
2024-11	7
2024-12	3
2025-01	6
2025-02	5
2025-03	47
2025-04	25

#	Time	Action	Message	Duration / Fetch
5	13:56:34	UPDATE customers SET signup_month = DATE_FORMAT(signup_date, '%M')	221 row(s) affected Rows matched: 221 Changed: 221 Warnings: 0	0.016 sec
6	13:58:02	SELECT DAYNAME(signup_date) AS signup_day, COUNT(*) AS gma_signup FROM customers WHERE is...	7 row(s) returned	0.000 sec / 0.000 sec
7	14:02:29	CREATE TABLE vip_customers AS SELECT * FROM customers WHERE city IN ('Delhi','Mumbai','Bangalore')...	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.062 sec
8	14:05:19	SELECT COUNT(*) AS vip_count FROM vip_customers LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
9	14:05:19	SELECT * FROM vip_customers LIMIT 20	20 row(s) returned	0.000 sec / 0.000 sec
10	14:08:09	SELECT DATE_FORMAT(signup_date, '%Y-%m') AS month, COUNT(*) AS signup_count FROM customers W...	7 row(s) returned	0.015 sec / 0.000 sec

- Get a list of cities with more than 20 customers.

```
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city
HAVING COUNT(*) > 20
ORDER BY customer_count DESC;
```

SQL File 4*

```

107
108 -- 3.2. Cities with > 20 customers
109 SELECT city, COUNT(*) AS customer_count
110 FROM customers
111 GROUP BY city
112 HAVING COUNT(*) > 20
113 ORDER BY customer_count DESC

```

city	customer_count
Kolkata	32
Pune	32
Bangalore	31
Delhi	31
Hyderabad	28
Ahmedabad	26
Mumbai	22

Result 5 x

#	Time	Action	Message	Duration / Fetch
6	13:58:02	SELECT DAYNAME(signup_date) AS signup_dow, COUNT(*) AS gmail_signups FROM customers WHERE is...	7 row(s) returned	0.000 sec / 0.000 sec
7	14:02:29	CREATE TABLE vip_customers AS SELECT * FROM customers WHERE city IN ('Delhi','Mumbai','Bangalore') ...	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.062 sec
8	14:05:19	SELECT COUNT(*) AS vip_count FROM vip_customers LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
9	14:05:19	SELECT * FROM vip_customers LIMIT 20	20 row(s) returned	0.000 sec / 0.000 sec
10	14:08:09	SELECT DATE_FORMAT(signup_date, '%Y-%m') AS month, COUNT(*) AS signup_count FROM customers W...	7 row(s) returned	0.015 sec / 0.000 sec
11	14:09:18	SELECT city, COUNT(*) AS customer_count FROM customers GROUP BY city HAVING COUNT(*) > 20 ORD...	7 row(s) returned	0.000 sec / 0.000 sec

Query Completed

➤ Find the date with the highest number of signups.

```

SELECT signup_date, COUNT(*) AS signup_count
FROM customers
GROUP BY signup_date
ORDER BY signup_count DESC, signup_date DESC
LIMIT 1;

```

SQL File 4*

```

114
115 -- A3. Date with the highest number of signups
116 SELECT signup_date, COUNT(*) AS signup_count
117 FROM customers
118 GROUP BY signup_date
119 ORDER BY signup_count DESC, signup_date DESC
120 LIMIT 1;

```

signup_date	signup_count
2025-03-31	5

Result 6 x

#	Time	Action	Message	Duration / Fetch
7	14:02:29	CREATE TABLE vip_customers AS SELECT * FROM customers WHERE city IN ('Delhi','Mumbai','Bangalore') ...	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.062 sec
8	14:05:19	SELECT COUNT(*) AS vip_count FROM vip_customers LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
9	14:05:19	SELECT * FROM vip_customers LIMIT 20	20 row(s) returned	0.000 sec / 0.000 sec
10	14:08:09	SELECT DATE_FORMAT(signup_date, '%Y-%m') AS month, COUNT(*) AS signup_count FROM customers W...	7 row(s) returned	0.015 sec / 0.000 sec
11	14:09:18	SELECT city, COUNT(*) AS customer_count FROM customers GROUP BY city HAVING COUNT(*) > 20 ORD...	7 row(s) returned	0.000 sec / 0.000 sec
12	14:10:28	SELECT signup_date, COUNT(*) AS signup_count FROM customers GROUP BY signup_date ORDER BY sig...	1 row(s) returned	0.000 sec / 0.000 sec

Query Completed

➤ Add a new column to show the day of the signup date. Find the day with the highest number of signups.

```

ALTER TABLE customers
ADD COLUMN signup_day VARCHAR(20);

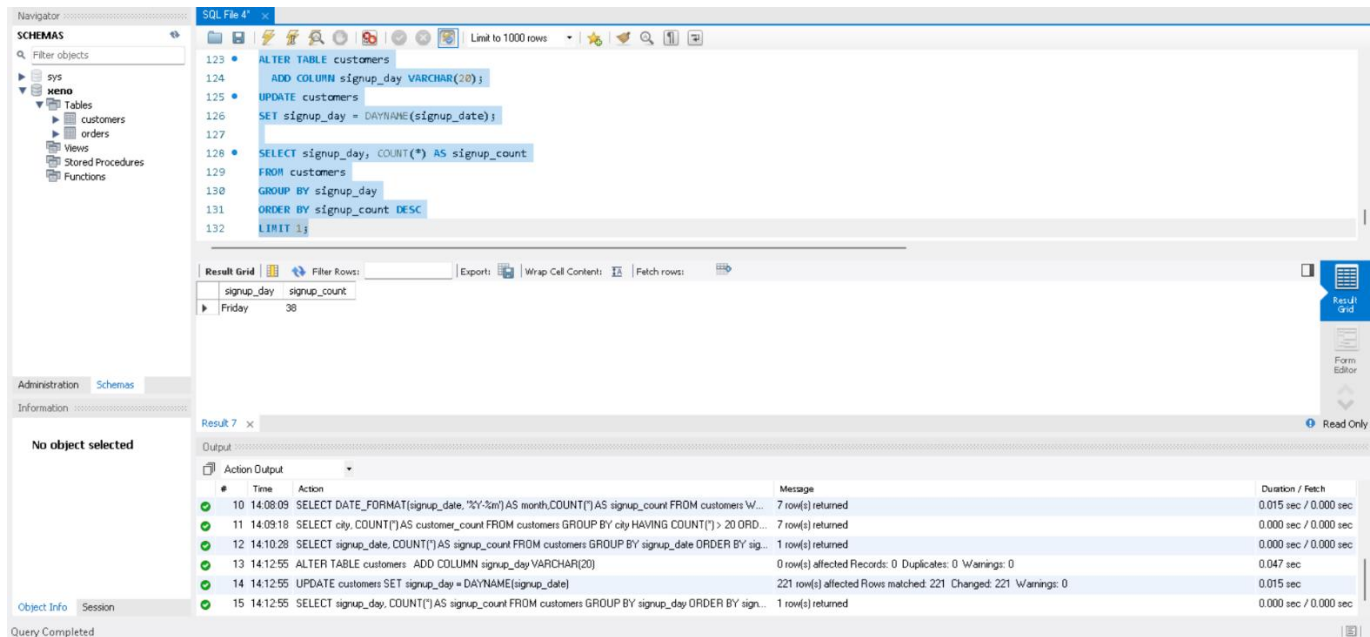
```

```

UPDATE customers
SET signup_day = DAYNAME(signup_date);

SELECT signup_day, COUNT(*) AS signup_count
FROM customers
GROUP BY signup_day
ORDER BY signup_count DESC
LIMIT 1;

```



Use Of AI in my Project:

I completed the assignment primarily by working hands-on in MySQL Workbench, writing and executing all SQL queries myself. I reviewed the dataset structure, validated columns, imported the CSV, resolved the DD-MM-YYYY date format, and performed all transformations and analysis directly using SQL. I used ChatGPT only as a reference for **non-functional steps**, such as clarifying how to:

1. Turn off safe-update mode,
2. Reconnect a MySQL Workbench connection (for example: opening MySQL Workbench, selecting "+" to add a new connection, entering hostname/user/password, testing the connection, and saving it).
3. Importing Data into the Table in MySQL